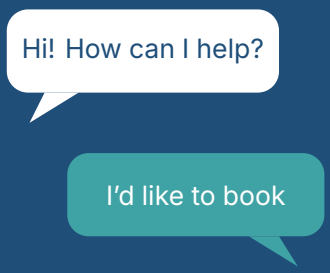
Building agents with keelchat



a plain guide

How to make chatbots for support, bookings, surveys, documents and data with keelchat, explained step by step for people who are new to chatbots.

Inside: what keelchat is • how a message flows • setting up • your first bot • six recipes • making bots behave • testing • extending with your own code • going live • troubleshooting

November 2026

keelchat is open source (Apache-2.0)

Contents

1 What keelchat is, in one page **2**

2 How it works: two pictures **3**
2.1 The life of one message . . . 3
2.2 The deployment picture . . . 4

3 Getting set up **5**
3.1 In the browser . . . 5
3.2 On your computer . . . 5
3.3 As a Python package . . . 5
3.4 The model key, in `.env` . . . 6
3.5 Deploy to Render . . . 6

4 Your first bot **6**
4.1 The two-line bot . . . 6
4.2 Build it . . . 7
4.3 Try it in the console . . . 7
4.4 Try it through the API . . . 7

5 Recipes: one per kind of bot **8**
5.1 Support assistant: collect an order number, then hand over . . . 8
5.2 Appointment booking with an emergency rule . . . 9
5.3 Customer survey (NPS) . . . 10
5.4 Answering from your documents . . . 12
5.4.1 Step 1: make the knowledge base . . . 12
5.4.2 Step 2: the bot . . . 12
5.5 Data questions over a table . . . 13
5.6 A bot that calls your own tool . . . 14

6 Making bots behave **15**

7 Testing your bot **17**
7.1 In the console . . . 18
7.2 From the command line . . . 18

8 Extending keelchat with your own code **18**
8.1 Replacing parts of knowledge search . . . 19

9 Going live **20**
9.1 Channels and Publish . . . 20
9.2 API keys . . . 21
9.3 Checklist before real customers . . . 21

10 Troubleshooting **22**

1 What keelchat is, in one page

How to read this guide Sections 1–4 are the basics: read them in order. Section 5 has one recipe per kind of bot; jump to the one you need. Sections 6–10 are for when you want more control, want to test properly, or are ready to go live. Every YAML example in this guide was checked with `keelchat build`, so you can copy it as it is.

keelchat is an open-source framework for **chatbots that talk to your customers**: booking an appointment, handling a support request, filling in a form, running a survey. A *framework* is a ready-made base you build on, so you write only the part that is special to you.

Three ideas explain almost everything:

1. **A bot is one YAML file.** YAML is a simple text format of `name: value` lines. The file says what the bot is for, what it asks, which rules it follows and when it passes the chat to a person. keelchat calls this file the bot's **policy**.
2. **One runtime runs every bot.** The *runtime* is the program that receives messages and sends replies. There is no special code per bot: change the YAML and you change the bot.
3. **The model reads and words replies; code decides.** The *model* is the AI language model (for example Claude). It understands what people write and phrases the answers. But the decisions – which question comes next, whether an answer is good enough, how many tries are allowed, when to hand over to a person – are made by keelchat's code, following your policy. A model never ends a conversation or skips a step on its own.

Why does the third point matter? A bot that only has instructions in a prompt can *drift*: it skips questions, loops, or says "I'll connect you to someone" and never does. In keelchat those things are counted and enforced in code, so they happen every time and you can test them.

You don't have to write YAML In the console's **Compose** view, or in the browser demo, you describe the bot in plain English. keelchat writes the YAML, reads it back to you in plain words, and lets you try it. This guide shows the YAML so you understand what is going on underneath.

Glossary: the words you will meet

Policy	The YAML file that defines one bot. It is checked and compiled before it runs.
Charter	Two or three sentences in the policy: what the bot helps with, its tone, and what is out of scope. Every prompt includes it.
Step	One part of the conversation, such as "greet and ask consent" or "fill in a form". Steps are listed under `flow`.
Rule	"*When* the customer says X, *then* do Y." Applies only on messages that fit.
Tool	A function the bot may call to look something up, such as an order status or free time slots. Tools are called only through the *Tool Gateway*.
Guardrail	An automatic check on what users send (input) or what the bot replies (output), for example "remove personal details".
Handoff	Passing the conversation to a person. It lands in the handoff inbox and can be sent to your help desk.
Knowledge base	A named set of your documents (help pages, manuals, FAQs) that a bot answers from.
Channel	Where people talk to the bot: website widget, WhatsApp, SMS, Slack, phone.
Console	keelchat's web app for building, trying and testing bots.

2 How it works: two pictures

2.1 The life of one message

Every message goes through three stages: **Understand**, **Decide**, **Respond**. The model helps in the first and last; the middle is your policy, enforced in code.

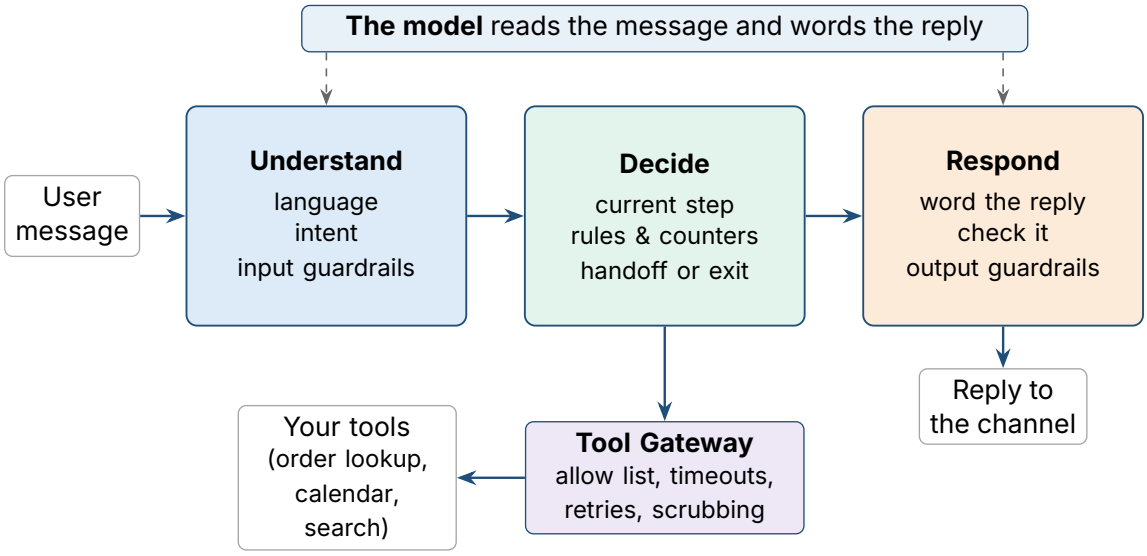


Figure 1: Understand → Decide → Respond. Tools are reached only through the Tool Gateway.

Understand
keelchat detects the language, runs the input guardrails (for example `prompt_injection`, which blocks attempts to trick the bot), and works out what the person meant: a new answer,

a change to an earlier answer, a side question, or something off-topic.

Decide
The current step of your `flow` runs. Answers are checked against what you said to `expect`. Re-asks, side trips and failures are counted against your `limits`. Rules that fit are added. If a tool is needed, it is called through the **Tool Gateway**, which only allows tools the policy lists, applies timeouts and retries, and treats tool output as data, never as instructions.

Respond
The model words the reply in the charter's tone. A checker makes sure nothing was invented, the output guardrails run (`pii` always removes personal details that should not be shown), and the channel shapes the message (for example SMS splits it into 160-character pieces).

2.2 The deployment picture

"Deployment" means where the pieces run. In the simplest setup it is all *one* server process; the extra services are optional and only needed when you grow.

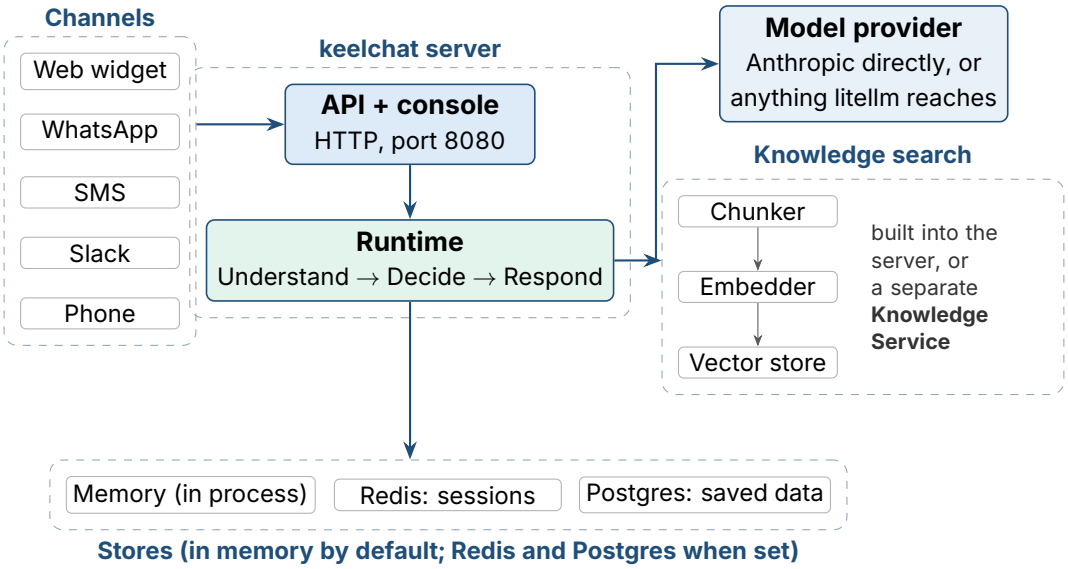


Figure 2: Channels talk to the API; the runtime calls the model provider, knowledge search and the stores.

- **Channels** turn each platform's messages into one common form, and each channel owns its own limits and rules (for example SMS answers `STOP` before the bot sees it).
- **API + console**: one process serves both the HTTP API and the console web app.
- **Model provider**: keelchat calls Anthropic directly, or any provider that *litellm* (a library that speaks to many AI providers) reaches: OpenAI, Gemini, Bedrock, OpenRouter, Ollama and more.
- **Knowledge search**: documents are split into passages (*chunker*), turned into lists of numbers that capture meaning (*embedder*), and kept where they can be searched by meaning (*vector store*). The server does this itself; for large sets you run the separate Knowledge Service.
- **Stores**: where state is kept. With nothing set, everything is in memory and is lost when the server stops. Redis keeps sessions; Postgres keeps memory, saved bot versions,

datasets, the audit log and handoffs.

3 Getting set up

Pick one of these paths. You can start with the first and move on later.

Path	Time	You need
Try it in your browser	1 minute	Nothing
Run it on your computer	5 minutes	macOS, Linux or WSL 2 (Linux inside Windows), and a model key
Install the Python package	5 minutes	Python 3.12 or newer, and a model key
Deploy to Render	10 minutes	GitHub and Render accounts, and a model key

3.1 In the browser

Open `https://keelchat-demo.fly.dev`. No sign-in is needed. Describe a bot, click through a few questions about what your description leaves open (emergencies, what to collect, languages), and you get its YAML. Press **Try this bot** to chat with it; its tools answer with sample data. A link can even start a draft:

```
https://keelchat-demo.fly.dev/?describe=Book+haircuts+and+ask+for+a+phone+number
```

3.2 On your computer

```
git clone https://github.com/pnrajan/keelchat && cd keelchat
./setup.sh        # checks for git, uv, Node 20+; offers to install what's missing;
                  # asks for your model key and writes it to .env
scripts/try.sh    # opens the console with a sample bot, everything in memory
```

`uv` is a fast tool that installs Python and the project's libraries for you, so you don't need to manage a "virtual environment" by hand. `scripts/try.sh` opens `http://127.0.0.1:8000/console/`; in the console, open **Compose**, describe a bot, press **Create my bot**, then **Try it**.

Other ways to run it locally:

- `./run.sh` starts the console, the API and the Knowledge and Analytics Services, with sample documents and sample data. Use it for the document and data recipes.
- `scripts/local.sh up` runs the whole stack in Docker (API, console, Redis, Postgres), and keeps your data between restarts.

3.3 As a Python package

keelchat is on PyPI, the public index of Python packages. The `[api]` part is an *extra*: it also installs what the web server needs.

```
pip install "keelchat[api]"      # needs Python 3.12 or newer
keelchat build my_bot.yaml       # check a policy
keelchat serve --port 8000       # run the API
```

In a checkout of the repository you put `uv run` in front of each command (`uv run keelchat build my_bot.yaml`); with `pip install` the command is simply `keelchat`.

3.4 The model key, in `.env`

keelchat reads its settings from environment variables. It loads them from a file named `.env`, which `./setup.sh` writes for you; `.env.example` lists the options. The minimum is a provider and its key:

```
# Anthropic directly
KEELCHAT_LLM=anthropic
ANTHROPIC_API_KEY=sk-ant-...

# or anything litellm reaches (also set the two model tiers and that provider's key)
# KEELCHAT_LLM=litellm
# KEELCHAT_MODEL_GENERATE=...    # writes replies
# KEELCHAT_MODEL_CLASSIFY=...    # reads messages, checks answers
# OPENROUTER_API_KEY=...
```

There is no pretend model keelchat always calls a real model, even in tests. A missing key stops it with an error instead of quietly using a fake. That way, what passes in testing is what runs for real.

3.5 Deploy to Render

Render is a hosting service with a free plan.

1. **Fork** the keelchat repository on GitHub (make your own copy), so the deploy follows your copy.
2. Open `https://render.com/deploy?repo=https://github.com/<you>/keelchat`.
3. Sign in, review the service, enter your `ANTHROPIC_API_KEY` when asked, and approve. The first build takes a few minutes.
4. On the service's **Environment** page, copy `KEELCHAT_API_KEY`. Render generated it; it locks the console and the API.
5. Open `https://<service>.onrender.com`. The console asks for the key once.

Saved bots need a database Without `DATABASE_URL` (and `REDIS_URL`), everything is in memory and is lost when the service restarts or sleeps. Add a Postgres URL on the **Environment** page. The free plan also sleeps when idle, so the first request after a quiet spell is slow.

4 Your first bot

4.1 The two-line bot

The smallest bot has just two keys. Everything else has a sensible default.

```
agent: shop-helper
charter: Help customers with orders and returns.
```

- `agent` is the bot's id: lowercase letters, digits and dashes.
- `charter` says what it helps with. Good charters also say the tone and what is out of scope.

With no `flow`, this bot simply chats within its charter. The default guardrails still run: prompt injection is blocked and personal details are removed from replies.

4.2 Build it

"Building" checks the file and compiles it into the form the runtime uses. Save the two lines as `my_bot.yaml` and run:

```
uv run keelchat build my_bot.yaml
```

If the file is fine, it prints the compiled policy. If not, every error names the exact place in the YAML. For example, a rule with `handoff: true` but no top-level `handoff` key gives:

```
policy invalid:
  rules[0].handoff: a rule that hands off needs a top-level `handoff:`
    (e.g. `handoff: {when: [user_request]}`)
```

`rules[0]` means "the first rule" (counting starts at 0).

4.3 Try it in the console

1. Start the console (`scripts/try.sh`).
2. Open **Editor · test chat**, paste the YAML and press **Build & try**. The editor completes keys, shows docs when you hover, and marks errors on their line as you type.
3. Chat with the bot in the pane beside it. The **Inspector** on the right shows *why* each reply was chosen: the step, the rule that fired, and how the reply was planned.
4. Happy with it? Press **Save version**. Only valid builds can be saved, and each one gets a number so you can compare and roll back.

Or skip the YAML: in **Compose**, say what the bot should do in a sentence, read what it will do in plain words, and ask for changes the same way ("also ask for their email").

4.4 Try it through the API

The console uses an HTTP API, and so can your own programs. Three calls make a conversation. Here they are with `curl` (a command-line tool that sends web requests) and `jq` (a tool that reads JSON):

```
API=http://127.0.0.1:8000

# 1. Build the policy (errors come back as YAML paths, like keelchat build)
curl -s $API/v1/agents -H 'content-type: application/json' \
  -d "$(jq -n --rawfile y my_bot.yaml '{yaml: $y}')"

# 2. Open a session (one conversation) with that bot
SID=$(curl -s $API/v1/sessions -H 'content-type: application/json' \
  -d '{"agent": "shop-helper", "user_id": "dev"}' | jq -r .id)

# 3. Send a message and read the reply
```

```
curl -s $API/v1/sessions/$SID/turns -H 'content-type: application/json' \
  -d '{"text": "Hi, where is my order?"}' | jq .outbound.text
```

> **Good to know** If the server has `KEELCHAT_API_KEY` set, add `-H "Authorization: Bearer $KEELCHAT_API_KEY"` to each call. Interactive API docs are at `/docs` on the server. A *session* stays on the policy version it started with, so changing a bot never changes a chat that is already going.

5 Recipes: one per kind of bot

Each recipe gives the YAML (built and checked), explains each part, and says how to try it.

5.1 Support assistant: collect an order number, then hand over

Good for: order problems, returns, account questions.
Main parts: `form_fill` step, a rule, guardrails, limits, `handoff`.

```yaml
agent: shop-support
charter: >
  Help customers of an online shoe store with orders and returns.
  Friendly and short. Out of scope: legal advice, anything not about the store.
languages: [en]
flow:
  - greet_consent
  - form_fill:
      fields:
        - name: order_number
          ask: What is your order number?
          expect: An order number in any format, such as ORD-12345 or #98765
        - name: issue
          ask: What went wrong with the order?
          expect: A short description of the problem
      closing: Thanks! Our team will email you within one working day.
rules:
  - name: damaged_item
    when: the customer says an item arrived damaged or broken
    then: >
      say a free replacement can be sent, and ask for the order number
      if it is not known yet
guardrails:
  input: [prompt_injection]
  output: [pii, no_legal_advice]
limits: {reasks: 3, digressions: 3}
handoff:
  when: [user_request, max_reasks]
```

What each part does

- `greet_consent` greets the person and asks if it may use and remember their answers.
- `form_fill` asks one field at a time. `ask` is a *sample* question: the model words it to fit the conversation. `expect` is what counts as a valid answer, written as you would explain it to

a person, not as a pattern. Answers that don't fit are asked again. `closing` is said once every field is answered.

- The `damaged_item` rule adds its `then` to the reply only on a message that fits its `when`.
- `guardrails`: `prompt_injection` and `pii` always run; `no_legal_advice` is added because the charter rules out legal advice.
- `limits`: at most 3 re-asks of one question, and 3 off-topic turns in a row before it steers back.
- `handoff`: the chat goes to a person when they ask (`user_request`) or when an answer fails too often (`max_reasks`).

How to try it. Build & try, then: "hi", "yes", "my shoes arrived broken" (the rule should fire, and the Inspector shows it), "ORD-12345", then "the sole came off". Start again and type "can I talk to a human?" to see the handoff. Handed-over chats appear in the console's **Handoffs** tab.

> **Make the bot more careful** The repository's `examples/support_assistant.yaml` adds `think: true` (plan each reply first) and `curious: true` (one follow-up when an answer is too thin, such as "it's broken"). Both cost an extra model call, so they are off by default. See Section 6.

5.2 Appointment booking with an emergency rule

Good for: clinics, salons, any "book a slot" service where some messages must go to a person at once.
Main parts: `form_fill`, a rule with `handoff: true`, a top-level `handoff`.

```
agent: dental-booking
charter: >
  Book check-ups and cleanings for Bright Smile Dental. Warm and brief.
  Out of scope: medical advice, prices, anything not about appointments.
languages: [en, es]
flow:
  - greet_consent
  - form_fill:
      fields:
        - name: treatment
          ask: Is this for a check-up or a cleaning?
          expect: one of check-up or cleaning
        - name: preferred_day
          ask: Which day suits you best?
          expect: a day of the week or a date
        - name: full_name
          ask: What name should the booking be under?
          expect: a person's name
        - name: phone
          ask: What phone number can we reach you on?
          expect: a phone number
      closing: Thank you! The front desk will text you to confirm the time.
rules:
  - name: dental_emergency
```

```
    when: >
      the patient describes severe pain, heavy bleeding, swelling,
      or a knocked-out or broken tooth
    then: say this sounds urgent and that a person from the clinic will take over
    handoff: true
guardrails:
  output: [pii, no_medical_advice]
handoff:
  when: [user_request]
```

What each part does

- The four fields collect what the front desk needs. `languages` lists English and Spanish; the bot detects the language, confirms before switching, and answers in it.
- The `dental_emergency` rule has `handoff: true`. When it matches, the conversation passes to a person *in code*: the reply says so and asks nothing more. The handoff's reason is `rule:dental_emergency`.
- `handoff: true` on a rule **needs** the top-level `handoff` key, or the build fails (you saw that error in Section 4).
- `no_medical_advice` matches the charter's "Out of scope: medical advice".

Why this matters. When the emergency instruction only lives in a prompt, a bot might say "I need to connect you with a staff member" and then carry on asking which treatment you want, with nobody getting the chat. With the rule in code, a person gets it every time.

How to try it. Start booking, then write "I just knocked a tooth out playing football". The reply should say a person is taking over, and the chat should appear in **Handoffs**.

> **Booking real slots** This bot collects a preferred day; the front desk confirms the time. To offer real free slots, you add a tool and a step of your own. The repository's `examples/booking_assistant.yaml` does this with an extension package (a `slot_search` tool and a `pick_slot` step). Recipe 5.6 shows how to write a tool.

5.3 Customer survey (NPS)

Good for: after-purchase feedback, satisfaction scores, short questionnaires.
Main parts: `survey` step with questions of different kinds; `ask_if` for follow-ups.

NPS (Net Promoter Score) is the classic "How likely are you to recommend us, from 0 to 10?" question.

```
agent: store-survey
charter: >
  Run a short survey after an order. Warm, brief, one question at a time.
  Out of scope: refunds and order changes; for those, say a person will follow up.
languages: [en]
flow:
  - greet_consent
  - survey:
      questions:
        - key: nps
```

```
          kind: scale
          prompt: How likely are you to recommend us to a friend?
          scale_min: 0
          scale_max: 10
          min_label: not at all
          max_label: extremely likely
        - key: what_went_wrong
          prompt: What could we have done better?
          expectation: what went wrong and how it affected them
          max_probes: 2
          ask_if: {question: nps, op: lte, value: "6"}
        - key: best_part
          prompt: What did you like most about your order?
          ask_if: {question: nps, op: gte, value: "7"}
        - key: channel
          kind: single_choice
          prompt: How did you first hear about us?
          options: [Search engine, Social media, A friend, Other]
      closing: Thank you, that really helps us.
handoff:
  when: [user_request]
```

What each part does

- `survey` asks its `questions` in order and records each answer and how it was captured.
- The `key` `nps` makes the answer an NPS score. `kind: scale` with `scale_min`/`scale_max` and labels for the ends. Closed kinds (`scale`, `single_choice`, `multi_choice`, `rank`, `allocation`, `number`) are read in code; a messy reply ("my sister told me") is interpreted and read back for the person to confirm.
- Open questions (the default `kind`) are conversational: `expectation` says what a complete answer covers, and the bot asks up to `max_probes` follow-ups until it does.
- `ask_if` asks a question only when an earlier answer meets a condition, decided in code. Here, a score of 6 or less gets "What could we have done better?" and 7 or more gets "What did you like most?".

Prefer a natural chat over fixed questions? `conversation_survey` takes *objectives* (what you want to learn) and finds each out in a natural conversation, skipping what the person already said:

```
agent: order-feedback
charter: >
  Learn how a customer's recent order went, in a short, warm conversation.
  Out of scope: refunds and order changes; for those, say a person will follow up.
flow:
  - greet_consent
  - conversation_survey:
      objectives:
        - key: nps
          learn: How likely they are to recommend us to a friend
          measure: nps
        - key: experience
          learn: What stood out about their order, good or bad, and why
```

```
          max_probes: 1
        - key: improve
          learn: The one thing we could do better next time
      max_questions: 6
```

How to try it and see results. Answer it a few times with low and high scores. Finished surveys are kept for findings: `GET /v1/tenants/{t}/agents/{agent}/findings` gives counts and NPS (computed in code) and themes with real quotes; `.../responses.csv` downloads the answers. The console's **Publish · results** shows scores over time. A shared link with `?score=8` starts the survey with a score already clicked, which is handy in an email.

5.4 Answering from your documents

Good for: help centres, product manuals, FAQs, policies.
Main parts: a knowledge base and a `knowledge` step with `dataset`.

A **knowledge base** is a named set of your documents. The bot answers *only* from passages it finds there, and each answer is checked against them. When the documents don't cover a question, the bot says so instead of guessing.

Step 1: make the knowledge base

In the console: Compose → **Knowledge base**. Upload files (PDF, text, Markdown, CSV, HTML) or add web pages, press **Create knowledge base**, then apply the change it proposes. Or use Publish → *Create a chatbot from your content* to make a whole bot from documents in one go.

By API: send files and web addresses as a form upload (`-F` sends one form field):

```
curl -s -X POST $API/v1/tenants/default/documents \
  -F agent=help-centre -F name=help_centre \
  -F files=@returns-policy.pdf -F files=@delivery.md \
  -F urls=https://example.com/help/warranty
# -> {"dataset": "help_centre", "documents": [...], ...}
#    plus the indexing method it picked, and why
```

A *tenant* is one customer account on a keelchat server; on your own server it is `default`. `name` names the knowledge base (default `kb_<agent>`). Documents already in a folder or S3 can be registered with `PUT /v1/tenants/{t}/datasets/{name}`.

Step 2: the bot

```
agent: help-centre
charter: >
  Answer customers' questions about delivery, returns and warranties
  from our help centre. Short, friendly answers.
  Out of scope: anything the help centre does not cover.
languages: [en]
flow:
  - knowledge:
      dataset: help_centre
handoff:
```

```
  when: [user_request]
```

The `knowledge` step's `dataset` names the knowledge base. Several bots can share one.

Side questions in another bot. A bot with a form can also answer side questions from documents: list the `knowledge_search` tool under `tools`. The question is answered and the open question is asked again.

```
agent: support-plus
charter: >
  Help customers with orders and returns. Friendly, concise.
  Out of scope: legal advice.
tools: [knowledge_search]
flow:
  - greet_consent
  - form_fill:
      fields:
        - name: order_number
          ask: What is your order number?
          expect: An order number in any format
handoff: {when: [user_request]}
```

How it's indexed. "Indexing" prepares the documents for search. With the default (`auto`), keelchat picks the method and tells you why:

Documents	Method	What it does
Fit in one prompt (about 40,000 characters)	`full_context`	The bot reads every document whole
Long documents (a manual)	`raptor`	Summaries at several levels, for broad and detailed questions
Many shorter documents	`contextual`	Each passage gets a line of context so search finds the right one
No model to index with	`simple`	Passages found by meaning

How to try it. `./run.sh` starts the Knowledge Service with sample documents; otherwise the server indexes knowledge bases itself. Ask something the documents answer, then something they don't: the bot should say it doesn't know.

> **Watch out** `keelchat build` on your own computer checks the YAML's shape. Whether the dataset *exists* is checked when you build on the server, where the knowledge base lives. Make the knowledge base first.

5.5 Data questions over a table

Good for: "Which region sold most?" over your own sales or operations data.
Main parts: a structured dataset and an `analytics` step.

```
agent: sales-analyst
charter: >-
  Answer questions about the online store's sales data. Every answer uses numbers
  from the data, states its assumptions and ends with practical recommendations.
  Out of scope: anything that is not about this sales data.
```

```
flow:
  - analytics:
      dataset: sales
limits: {digressions: 3, repairs: 2}
```

A **structured dataset** is one or more tables (CSV or Parquet files). Questions become SQL (the language for asking tables questions) run on a locked, in-memory copy of the data. Every answer is planned, checked and verified before it is shown; numbers must come from the result. An answer that fails its checks is marked *partial*. `limits.repairs` is how many times a step may fix its own failed attempt (for example rewrite failing SQL).

Register the dataset first. Sources must sit under `KEELCHAT_DATA_ROOT`:

```
curl -s -X PUT $API/v1/tenants/default/datasets/sales \
  -H 'content-type: application/json' -d '{
  "description": "Online store orders, January to June 2026",
  "spec": {"kind": "structured", "tables": [
    {"name": "orders", "uri": "'"$PWD"'/examples/data/orders.csv",
     "columns": {"amount": "order total in USD"}},
    {"name": "targets", "uri": "'"$PWD"'/examples/data/targets.parquet",
     "description": "Quarterly revenue target per region"}]}}'
```

How to try it. `./run.sh` (or `scripts/dev.sh --analytics`) starts the Analytics Service and registers the sample data as `sales` for you. Ask “Which region brought in the most revenue?” and then a follow-up like “and in March?”. Statistics code (beyond SQL) runs only in a sandbox with no network; turn it on with `./run.sh --sandbox`.

5.6 A bot that calls your own tool

Good for: looking things up in your own systems: order status, stock, opening hours.
Main parts: a Python function with `@tool`, listed in `tools`.

A **tool** is an `async` Python function (one that can wait for a network reply without blocking). keelchat turns its type hints into the input schema, and the *docstring* (the text in triple quotes) is what the model reads to decide when to use it.

```
from keelchat import ToolError, tool

ORDERS = {"ORD-12345": "shipped", "ORD-67890": "processing"}


@tool("order_status", timeout_s=3.0, retries=1)
async def order_status(order_number: str) -> dict[str, str]:
    """The delivery status of one order, looked up by its order number."""
    status = ORDERS.get(order_number.strip().upper())
    if status is None:
        raise ToolError("order_status", "not_found", "no such order")
    return {"order_number": order_number, "status": status}
```

- `timeout_s=3.0`: the gateway stops waiting after 3 seconds. `retries=1`: it tries once more after a timeout or error; after repeated failures a circuit breaker stops calling the tool for a while.

- Raise `ToolError(name, kind, message)` for failures. A dictionary stands in for your real system here.
- Tools only *read*. Anything that changes something (book, pay, submit) must be an `@action` instead, which runs only on the user's own explicit action.

Now allow it in a policy:

```
agent: order-tracker
charter: >
  Tell customers where their order is. Short and friendly.
  Out of scope: refunds, payments, anything not about deliveries.
mode: planner
tools: [order_status]
limits: {planner_calls: 3}
handoff:
  when: [user_request, tool_unavailable]
```

`mode: planner` means there is no fixed list of steps: each turn the *planner* picks read-only tools from the allow list and answers from what they return. `limits.planner_calls` caps tool calls per turn, counted in code. `tool_unavailable` in `handoff` passes the chat to a person if the tool is down.

Make keelchat find your code. Put the file in a package folder (here `my_tools/` with an empty `__init__.py`) and name the package in `KEELCHAT_CAPABILITY_PACKAGES`:

```
PYTHONPATH=. KEELCHAT_CAPABILITY_PACKAGES=my_tools \
  uv run keelchat build order_tracker.yaml
```

Without it the build stops with `tools[0]: unregistered tool 'order_status'` and lists the tools it knows.

> **Start from a template** `uv run keelchat new package acme_caps --out my_bot` writes a starter package with a tool, a step, a guardrail, an agent that uses them, and a passing test. It is the quickest way to see how the parts fit.

6 Making bots behave

Here is a policy that uses most of the "behaviour" settings at once. Each one is explained below.

```
agent: returns-desk
charter: >
  Help customers return items bought from our store. Friendly, concise.
  Out of scope: legal advice, medical advice.
languages: [en, es]
small_talk: brief
flow:
  - greet_consent: {model: small}
  - form_fill:
      think: true
      curious: true
      fields:
```

```
        - name: order_number
          ask: What is your order number?
          expect: An order number in any format
        - name: reason
          ask: Why are you returning it?
          expect: A short reason for the return
rules:
  - name: big_refund
    when: the customer asks for a refund over $500
    then: say a person will review large refunds today
    high_stakes: true
guardrails:
  input: [prompt_injection, sql_injection]
  output: [pii, profanity, no_legal_advice]
limits: {reasks: 2, digressions: 3, reopens: 2}
memory:
  - field: preferred_contact
    extract: How they prefer to be contacted (email, phone, chat)
    type: email|phone|chat
    half_life: 90d
  - field: home_address
    extract: Their home address, when they give it
    sensitive: true
    confirm_on_reuse: true
handoff:
  when: [user_request, max_reasks]
  timeout: 24h
```

Setting	What it does
`rules` (when/then)	Applies only to messages that fit `when`. Write `when` about the customer's message ("asks for a refund over $500"), not the mood of the chat. A match must quote the latest message, so an earlier message never fires a rule again. `overrides` settles conflicts between rules.
`limits`	Counted in code: `reasks` (per question, default 3), `digressions` (off-topic turns in a row, 3), `reopens` (edits to a finished step, 2), `planner_calls` (6), `on_step_enquiries` (3), `repairs` (2). SMS users tolerate fewer re-asks than web users.
`guardrails`	Input: `prompt_injection`, `sql_injection`. Output: `pii`, `profanity`, `no_legal_advice`, `no_medical_advice`. `prompt_injection` and `pii` always run.
`memory`	Only the items you list are remembered between conversations, each typed and fading by `half_life`. `sensitive: true` items need consent, must be volunteered, and never reach a tool. `confirm_on_reuse` asks "is it still ...?". Needs `greet_consent` first in the flow.
`languages`	Languages it speaks; the first is the fallback. List only ones you support end to end.
`small_talk`	`off` (default: chit-chat counts as off-topic), `brief` (one friendly line, then back), `warm` (one or two sentences). Never taken as an answer.
`think: true`	On a step: plan each reply first (what they need, what is unknown, which rules apply). The plan shows in the Inspector's **Why** tab. One more model call.

Setting	What it does
`curious: true`	On a `form_fill`: one follow-up when an answer is valid but too thin ("it's broken" – broken how?). Never for ids, numbers, names, dates or yes/no.
`model: small|large`	On a step: run all its work on one model size. Simple steps can use `small` to save money.
`high_stakes: true`	On a step or a rule: that turn's reply is written with the large model and always checked before sending.
`handoff`	`when`: `user_request`, `max_reasks`, `disengaged`, `tool_unavailable`. `notify`: `handoff_webhook` also sends it to your help desk (`HANDOFF_URL`). `timeout` gives the chat back to the bot if nobody replies in time.
`mode`	`flow` (every reply from a step), `adaptive` (a multi-part side question may use the planner), `planner` (no fixed steps).

> **Charter vs. settings** Put scope and tone in the charter. Don't put limits or guardrails in the charter text: they belong in their own keys, where code enforces them. Name the out-of-scope topics users actually bring up.

Durations are a number and a unit: `s`, `m`, `h`, `d` or `w` (`90d`, `24h`). Text YAML would read as true/false (`yes`, `no`, `on`, `off`) needs quotes: `"yes"`.

What happens on a handoff

1. **Triggered**: the user asks for a person, a rule with `handoff: true` matches, or a dead end in `when` is reached.
2. **Sent**: a ticket opens in the handoff inbox (console **Handoffs** tab) with the reason, the answers so far and the recent transcript, with sensitive values removed.
3. **Waiting**: later messages join the same ticket.
4. **Reported back**: a person presses **Resolve and resume** or **Resolve and end**, or your help desk calls the ticket's `callback_url`.
5. **Resumed**: the bot says it is back and carries on from the next open question.

7 Testing your bot

Test before real people use the bot. keelchat gives you three levels.

7.1 In the console

Where	What you do
Test → Tests	Save a chat as a test, draft tests from the policy, run them all on the current build. `scripts/try.sh` saves three sample tests: press **Run**.
Simulate	Model-played personas (pretend customers) and scripted replays, on fake tools.
Rate	Mark each reply in scope or not, each rule followed or not; let the model suggest policy changes.
Judge	A stronger model (`KEELCHAT_MODEL_CRITIC`) reads a trial chat, lists what went wrong on which turn, and proposes fixes. Tick the policy fixes and **Apply to the policy**; the build checks them and nothing is saved until you save.

7.2 From the command line

An *eval set* is a folder of test conversations for one bot, kept in `evals/<agent>/`.

```
uv run keelchat evals generate my_bot.yaml          # draft a test set
uv run keelchat evals run evals/support_assistant    # run it: checks done in code
uv run keelchat evals check evals/support_assistant  # also the judged checks
uv run keelchat simulate examples/hiring_agent.yaml examples/sim_script.yaml
```

A test case is plain YAML in `replay.yaml`: messages to send, and what must be true afterwards.

```
cases:
  - name: happy_path
    messages: ["hi", "yes", "ORD-12345", "it never arrived"]
    expect: {completed: true, fields: {order_number: ORD-12345}}
  - name: damaged_item_rule
    messages: ["hi", "yes", "my blender arrived broken", "ORD-12345"]
    expect: {rules: {3: [damaged_item], 4: []}}   # turn 3 must match the rule
  - name: wants_a_person
    persona: Agrees to share answers, then insists on talking to a real person.
    max_turns: 6
    expect: {exit_reason: handed_off}
```

What to cover The happy path, every exit (too many re-asks, a person asked for), one edit ("actually my order number is …"), one message with two intents, one off-topic message, one per language, and one per rule. The judged evals need `pip install "keelchat[evals]"` and a judge model (`EVALS_JUDGE_MODEL`).

8 Extending keelchat with your own code

Every extension is **one decorated Python function or class**, registered by name. You never change the runtime. Everything imports from the top level: `from keelchat import tool, node, guardrail, …`.

You add	How	Used in the policy as
Tool	`@tool("name")`	`tools: [name]`
Step	`@node("name")` on a `Node` class	a `flow` entry: `- name: {...}`
Guardrail	`@guardrail("name", stage="output")`	`guardrails: {output: [pii, name]}`
Action (commits)	`@action("name", by="candidate")`	runs only on the user's explicit action
MCP server tools	list in `KEELCHAT_MCP_SERVERS`	`tools: [alias.tool]`
HTTP endpoint	`POST /v1/tenants/{t}/tools`	`tools: [name]`

MCP (Model Context Protocol) is a standard way for programs to offer tools to AI models; any MCP server's tools can be used without writing code. keelchat finds your package through the setting `KEELCHAT_CAPABILITY_PACKAGES`, or through an entry point in your `pyproject.toml`. `check_capability` runs the *contract suite* (the checks every capability must pass) on one of yours.

8.1 Replacing parts of knowledge search

Knowledge search has four parts you can swap. Each has an interface (a base class), a decorator that registers your version by name, and a setting that picks it.

Part	Interface	Register / setting	Built in
Embedding model	`Embedder`: `dim`, `id`, `embed(texts)`	`@embedder` / `KNOWLEDGE_EMBEDDER`	litellm, chonkie, sentence-transformers, siliconflow, hash
Splitting into passages	`Chunker`: `chunk(text)`	`@chunker` / `KNOWLEDGE_CHUNKER`	chonkie, semantic, sentences
Indexing method	`Indexer`: `chunks(doc)`	`@indexer` / `KNOWLEDGE_STRATEGY`	simple, contextual, raptor, full_context
Vector store	`VectorIndex`: `upsert`, `search`, `delete_doc`, `count`	`@vector_store` / `KNOWLEDGE_VECTOR_STORE`	memory, lancedb, qdrant, chroma, pgvector

Here is a complete, tiny vector store that keeps vectors in a Python dictionary, with its test. It was run, and `check_vector_store` passed.

```python
import asyncio
import math

from keelchat import Chunk, Hit, VectorIndex, VectorStoreConfig, vector_store
from keelchat.testing import check_vector_store


def cosine(a: list[float], b: list[float]) -> float:
    dot = sum(x * y for x, y in zip(a, b, strict=True))
    na = math.sqrt(sum(x * x for x in a)) or 1.0
    nb = math.sqrt(sum(x * x for x in b)) or 1.0
    return dot / (na * nb)
```

```
@vector_store("dictstore")  # pick it with KNOWLEDGE_VECTOR_STORE=dictstore
class DictStore(VectorIndex):
    def __init__(self, cfg: VectorStoreConfig):
        self.rows: dict[str, tuple[Chunk, list[float]]] = {}

    async def upsert(self, chunks: list[Chunk], vectors: list[list[float]]) -> None:
        for chunk, vector in zip(chunks, vectors, strict=True):
            self.rows[chunk.id] = (chunk, vector)  # same id: replace

    async def search(self, vector: list[float], k: int = 5, max_level: int | None = None,
    ↪  dataset: str = "") -> list[Hit]:
        hits = [Hit(chunk=c, score=cosine(vector, v)) for c, v in self.rows.values() if c.
    ↪ dataset == dataset and (max_level is None or c.level <= max_level)]
        hits.sort(key=lambda h: h.score, reverse=True)
        return hits[:k]

    async def delete_doc(self, doc_id: str) -> None:
        for cid in [i for i, (c, _) in self.rows.items() if c.doc_id == doc_id]:
            del self.rows[cid]

    async def count(self, dataset: str | None = None) -> int:
        return sum(1 for c, _ in self.rows.values() if dataset is None or c.dataset ==
    ↪ dataset)


async def test_dict_store() -> None:
    cfg = VectorStoreConfig(uri=None, table="chunks_test", dim=0)
    await check_vector_store(DictStore(cfg))


if __name__ == "__main__":
    asyncio.run(test_dict_store())
    print("check_vector_store passed")
```

A vector store must keep datasets apart, honour `max_level`, replace a chunk upserted again with the same id, and return whole chunks with cosine similarity as the score. `check_vector_store` tests all of that on an empty store and names the rule that failed. Then pick it with `KNOWLEDGE_VECTOR_STORE=dictstore`.

> **Good to know** A real store would talk to a database; this one forgets everything on restart, like the built-in `memory` store. Switching embedders re-indexes the documents rather than mixing vectors that can't be compared.

9 Going live

9.1 Channels and Publish

The console's **Channels** tab says what to paste or configure for each one.

Channel	How it connects
Website widget	Publish the agent (`POST /v1/tenants/{t}/agents/{agent}/embeds`) and paste the one-line `<script>` it returns. Only the sites you list can show it.
Link and app	The same publish returns `app_url`: a full-screen chat that installs on a phone.
WhatsApp	WhatsApp Cloud API: `POST .../embeds/{id}/whatsapp` stores the number's token encrypted.
Phone	Point a Twilio number's voice webhook at `/v1/public/phone/embeds/{id}`.
SMS	Post Twilio's form fields as the turn's `payload`; long replies are split, STOP is answered.
Slack	Post the Slack event as the turn's `payload`; lists become Slack blocks.

9.2 API keys

`KEELCHAT_API_KEY` is the admin key. Give each website or messaging backend its own key with `POST /v1/tenants/{t}/keys`; each is shown once and stored as a hash, and reaches only its own tenant.

Scope	Allows
`build`	everything in its tenant
`read`	read-only access
`runtime`	sessions and turns only: the key for a website or messaging backend

9.3 Checklist before real customers

- ☐ A database is set: `DATABASE_URL` (Postgres) and `REDIS_URL`, so saved bots and chats survive restarts (`scripts/db.sh up` starts both locally).
- ☐ `KEELCHAT_API_KEY` is set, and the website uses a `runtime`-scoped key, not the admin key.
- ☐ The charter names the out-of-scope topics users actually bring up.
- ☐ Every step asks one thing; every `expect` reads like an instruction to a person.
- ☐ Limits suit the channel; `handoff` is set, and someone watches the Handoffs inbox (or `notify` sends to your help desk).
- ☐ Memory items are typed, have a half-life, and sensitive ones are marked.
- ☐ Anything that commits (book, pay, submit) is an action, not a tool.
- ☐ Tools point at real systems (not `KEELCHAT_TOOLS=fake`, which is for development only).
- ☐ `keelchat build` and `keelchat evals run` are green.
- ☐ Set `KEELCHAT_PUBLIC_URL` so links and help-desk callbacks have a full address.

After launch, the **Outcomes** dashboard shows how conversations ended, and the audit log (`GET /v1/tenants/{t}/audit`) records every decision. `keelchat evals export` turns a failed live conversation into a test case.

10 Troubleshooting

What you see	What to do
A reply says it can't do something "right now" ("I can't look that up right now…")	The tool's service is down or its URL is unset: start it (`./run.sh` starts Knowledge and Analytics), or run with `--fake-tools`.
"The API no longer has this policy" or "This session is no longer on the server"	The API restarted with in-memory state: **Build & try** again and start a new session, or set `DATABASE_URL` and `REDIS_URL`.
The API exits at start with a database error	`DATABASE_URL` names a database that doesn't exist: `scripts/db.sh up`, or leave it empty.
Save version is disabled	The editor found problems; only valid builds are saved. Fix the marked lines.
The console shows "API offline"	The API process is down: check the terminal running `./run.sh`.
`./setup.sh` stops on a missing tool	It lists what's missing and the command it would run: rerun with `--yes`, or install them yourself. On Windows, run it inside WSL 2.
`scripts/local.sh up` says Docker is not running	Start Docker Desktop (macOS, Windows) or `sudo systemctl start docker` (Linux).
`scripts/local.sh up`: the API container exits	Run `scripts/local.sh logs`; usually there is no model key in `.env`.
`keelchat build`: `unregistered tool`	Your package isn't loaded: set `PYTHONPATH` and `KEELCHAT_CAPABILITY_PACKAGES`.
`keelchat build`: `a rule that hands off needs a top-level `handoff:` `	Add `handoff: {when: [user_request]}`.

Every log line carries a `session_id`, so searching the logs for it follows one conversation from start to end.

Where to read more

In the repository's `docs/` folder: **WRITING_AN_AGENT.md** (writing and tuning a bot), **POLICY_LANGUAGE.md** (every policy key, its values and defaults), **FEATURES.md** (console, channels, memory, handoff, documents), **ADDING_A_CAPABILITY.md** (tools, steps, guardrails, channels, knowledge search parts), **EVALS.md** (testing), **REFERENCE.md** (CLI, API, settings, deployment) and **ARCHITECTURE.md** (how the runtime works). The `examples/` folder has bots for support, bookings, surveys, hiring, a hotel concierge and data questions.